

EN UNA FRASE Sabes qué es Spring Boot, qué le añade a Spring Framework y qué tres cosas cambian en el `pom.xml` respecto al Volumen 1. Tienes el entorno listo (Java 17+, Maven, IntelliJ) para crear el proyecto en el capítulo siguiente.

LO ESENCIAL

Spring Framework	Framework Java nacido en 2003 para IoC, DI y AOP: el framework crea y conecta los objetos, tú declaras qué necesitas
Spring Boot	Capa sobre Spring (2014) que toma las decisiones de configuración por ti; no es un framework nuevo
Convención sobre configuración	Spring Boot asume valores sensatos (servidor HTTP en el puerto 8080); solo cambias lo que difiere
Autoconfiguración	Detecta las dependencias del classpath y configura los beans necesarios (Jackson, H2...)
Servidor embebido	Tomcat va dentro del JAR: no instalas ningún servidor externo
Starter	Dependencia agrupada con versiones compatibles; <code>spring-boot-starter-web</code> trae Tomcat, Spring MVC y Jackson
<code><parent> spring-boot-starter-parent</code>	Hereda las versiones de todas las dependencias; por eso los starters van sin <code><version></code>
<code>spring-boot-maven-plugin</code>	Genera el fat JAR con <code>mvn package</code> y habilita <code>mvn spring-boot:run</code>
<code>@RestController</code>	<code>@Controller</code> + <code>@ResponseBody</code> : el valor de retorno se serializa como JSON
<code>@GetMapping("/libros")</code>	Enruta las peticiones GET de esa URL a un método
<code>application.properties</code>	Fichero de configuración; por ejemplo <code>server.port=9090</code>
Java 17+	Versión mínima que exige Spring Boot 3.x

ERRORES TÍPICOS

1 Unsupported class file major version 61

- ✓ tu JDK es anterior a Java 17. Comprueba `java -version`, instala un JDK 17+ y revisa el SDK en **File** → **Project Structure** → **SDK**.

2 'dependencies.dependency.version' ... is missing

- ✓ falta el bloque `<parent>`. Añádalo antes de `<dependencies>`.

3 Port 8080 was already in use

- ✓ pon `server.port=9090` en `application.properties`, o localiza el proceso con `lsof -i :8080` y deténlo con `kill -9 <PID>`.

4 Whitelabel Error Page en el navegador

- ✓ has usado `@Controller`, que espera el nombre de una vista. Para una API REST usa `@RestController`.

5 Could not find or load main class

- ✓ has lanzado la clase con `java` a secas y el classpath está incompleto. Usa `mvn spring-boot:run` en desarrollo o `java -jar` con el fat JAR.

EN EL PROYECTO BIBLIOTECA

Todavía no hay código: el capítulo fija el punto de partida (la aplicación de consola Maven del Volumen 1, con CSV y menús) y el destino (una API REST construida capítulo a capítulo). `Libro`, `Usuario` y `Prestamo` se reescribirán en versión reducida y `Biblioteca` renacerá como `BibliotecaService`. El proyecto Spring Boot se genera en el Capítulo 2.

■ CÓDIGO MÍNIMO Y PETICIONES

Las tres piezas que distinguen un `pom.xml` de Spring Boot: el `<parent>`, un starter sin versión y el plugin.

```
<parent>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-parent</artifactId>
  <version>3.3.0</version>
</parent>

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
  <!-- Sin <version>: la hereda del parent -->
</dependency>

<plugin>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
```

XML

```
java -version          # Java 17 o superior
mvn -version           # Maven 3.6 o superior
mvn spring-boot:run    # arranca la aplicación sin generar el JAR
mvn package            # genera el fat JAR, ejecutable con java -jar
lsof -i :8080          # quién ocupa el puerto 8080 (Linux o macOS)
```

TERMINAL

Un endpoint completo con Spring Boot (frente a unas 15 líneas con `HttpServer` en Java puro):

```
@RestController
public class LibroController {

    @GetMapping("/libros")
    public List<String> listar() {
        return List.of("Clean Code");
    }
}
```

JAVA